

PowerPoint Presentation Explanation Guide

Beginner-Friendly Detailed Explanations

This document provides detailed, beginner-friendly explanations for each slide in the Scope and Delimitation PowerPoint presentation. It explains **what** each concept is and **why** it's important, using simple language and real-world analogies.

Slide 1: Title Slide

What It Is

The title slide introduces the presentation topic: **Scope and Delimitation** - showing how different system components work together to achieve the project's goals.

Why It Matters

- **Scope** = What the system CAN do (the 9 capabilities)
- **Delimitation** = What the system CANNOT do (limitations and constraints)
- This presentation shows HOW each component helps achieve the scope items

Key Terms Explained

- **Child-Centric Wi-Fi Monitoring and Control System**: A system designed specifically for monitoring and controlling children's internet access
- **Learning Access Management**: Children must complete educational activities (quizzes/videos) to earn more internet time
- **Automated Reporting**: The system automatically tracks and reports internet usage

Slide 2: Network Topology

What It Is

This slide shows **how devices are physically connected** in the network - like a map showing how data travels from the internet to child devices.

Why It Matters

Understanding the network topology helps explain:

- How child devices connect to the internet
- Why the Raspberry Pi can control all child device traffic
- How the system isolates child devices from the parent network

Key Concepts Explained

Network Topology Diagram

What: A visual map showing the path: Internet → ISP Router → Raspberry Pi → Child Device

Why: Shows that ALL child device internet traffic must pass through the Raspberry Pi, giving the system complete control.

Real-World Analogy: Like a security checkpoint at an airport - everyone must pass through one point where they can be checked and controlled.

Network Address Translation (NAT) with MASQUERADE

What: A technology that allows multiple devices to share one internet connection by translating IP addresses.

How It Works:

1. Child device (192.168.4.2) wants to visit [google.com](https://www.google.com)
2. Request goes to Raspberry Pi (192.168.4.1)
3. Pi "translates" the request: "This is from 192.168.4.2, but I'll send it as if it's from me"
4. Internet sees the request coming from Pi's public IP
5. Response comes back to Pi, which forwards it to the correct child device

Why It's Needed:

- Child devices have private IPs (192.168.4.x) that the internet doesn't recognize
- The Pi has a public IP from the ISP router
- NAT allows child devices to access the internet through the Pi's connection

Real-World Analogy: Like a receptionist in an office building. When someone calls the main number, the receptionist (NAT) routes the call to the right person (device) inside the building.

Network Zones and Isolation

Zone 1: WAN (eth0) - The Internet Connection

- **What:** The Raspberry Pi's connection to the internet (via Ethernet cable to ISP router)
- **IP:** Gets assigned automatically by the ISP router (DHCP)
- **Purpose:** Provides internet access to the Pi itself

Zone 2: Child Network (wlan0) - The WiFi Network

- **What:** The WiFi network that child devices connect to
- **IP:** Fixed at 192.168.4.1 (the Pi is the gateway)
- **Purpose:** Isolated network where all child devices connect
- **SSID:** "Parental_WiFi" (the network name children see)

Why Two Zones?

- **Isolation:** Child devices are completely separate from the parent's network
- **Control:** All child traffic flows through the Pi, allowing monitoring and blocking
- **Security:** Parents' devices stay on their own network, unaffected

Traffic Flow Control

Outbound Traffic (Child → Internet):

Child Device → WiFi AP → Firewall Check → NAT Translation → Internet

What Happens:

1. Child device sends request
2. WiFi access point (wlan0) receives it
3. iptables firewall checks if device is blocked
4. NAT translates the IP address
5. Request goes to internet via Ethernet (eth0)

Inbound Traffic (Internet → Child):

Internet → NAT Reverse Translation → Firewall Check → WiFi AP → Child Device

Why This Matters: The system can block, monitor, or redirect traffic at multiple points in this flow.

Key Network Services

hostapd - WiFi Access Point Manager

- **What:** Software that creates the WiFi network
- **Why:** Without this, there's no WiFi network for children to connect to
- **What It Does:**
 - Creates the "Parental_WiFi" network
 - Manages which devices can connect
 - Handles WiFi authentication

dnsmasq - DHCP & DNS Server

- **What:** Two services in one:
 - **DHCP:** Automatically assigns IP addresses to child devices
 - **DNS:** Translates website names to IP addresses AND blocks domains
- **Why:**
 - DHCP: Without it, you'd have to manually set IP addresses on every device
 - DNS: Allows domain-level blocking (blocking [facebook.com](https://www.facebook.com) blocks all Facebook)
- **Real-World Analogy:**
 - DHCP = Hotel receptionist assigning room numbers
 - DNS = Phone book that also blocks certain numbers

iptables - Firewall & NAT

- **What:** Linux firewall that controls network traffic
- **Why:** Provides the actual blocking mechanism at the network level
- **What It Does:**
 - **FORWARD Chain:** Controls traffic between WiFi and Ethernet
 - **INPUT Chain:** Controls access to the Pi itself
 - **NAT Table:** Handles IP address translation
 - **MAC-based Rules:** Blocks/unblocks specific devices by their unique MAC address

Benefits of This Architecture

- ✓ **Network Isolation:** Child devices can't access parent network or devices
- ✓ **Centralized Control:** All child traffic goes through one point (the Pi)
- ✓ **Monitoring Capability:** System can see everything child devices do online
- ✓ **Security:** Multiple layers of protection (firewall, MAC filtering)
- ✓ **Scalability:** Can handle multiple child devices at once

Slide 3: Scope Item 1 - Website Monitoring and Blocking

What It Is

This slide explains how the system monitors which websites children visit and allows parents to block specific websites.

Why It Matters

Parents need to:

- Know what their children are viewing online
- Block inappropriate or distracting websites
- Monitor for safety and educational purposes

Key Concepts Explained

Blocking Types

URL-level Blocking

- **What:** Block a specific webpage (e.g., <https://facebook.com/specific-page>)
- **When to Use:** When you only want to block one page, not the entire website
- **Example:** Block a specific YouTube video but allow other YouTube videos

Domain-level Blocking

- **What:** Block an entire website and all its subdomains
- **When to Use:** When you want to completely block a website

- **Example:** Block `facebook.com` which also blocks `m.facebook.com` , `www.facebook.com` , etc.
- **How:** Uses wildcard blocking (`*.facebook.com`)

App-level Blocking

- **What:** Block a mobile app by blocking ALL related domains
- **When to Use:** For mobile apps that use many different domains
- **Example:** Blocking Facebook app blocks 30+ domains including:
 - `facebook.com`
 - `api.facebook.com`
 - `graph.facebook.com`
 - `cdn.facebook.com`
 - And many more...
- **Why:** Mobile apps often connect to multiple servers, so blocking just one domain doesn't work

How Components Work Together

Step 1: Child Device Accesses Website

- Child tries to visit a website (e.g., facebook.com)
- Device needs to know the IP address, so it asks DNS server

Step 2: dnsmasq (DNS Server) Intercepts

- **What:** dnsmasq receives the DNS query (request for IP address)
- **Checks:** Is this domain blocked for this device?
- **If Blocked:** Returns `127.0.0.1` (localhost - a fake address that goes nowhere)
- **If Not Blocked:** Returns the real IP address
- **Result:** Blocked websites can't be accessed because the device can't find them

Step 3: Laravel Application Records Access

- **Background Job:** `ParseNetworkLogs` runs every 10 minutes
- **What It Does:**
 - Reads network logs to see which websites were accessed
 - Records them in the database (`browsing_logs` table)
- **Why:** Parents can see browsing history in the dashboard

Step 4: MariaDB Database Stores Information

- **browsing_logs:** Records every website visit (domain, timestamp, device)
- **flagged_websites:** Websites parents want to monitor (not block, just watch)

- **blocked_websites:** Websites that are blocked
 - block_type : url, domain, or app
 - related_domains : For app-level blocking, stores all related domains

Step 5: Parent Dashboard

- Parents can:
 - View browsing history
 - Flag websites (mark for monitoring)
 - Block websites (choose URL, domain, or app-level)
 - See which websites were attempted but blocked

Why DNS Blocking Works

How DNS Works:

1. You type "facebook.com" in browser
2. Device asks DNS: "What's the IP address for facebook.com?"
3. DNS responds: "It's 31.13.64.35"
4. Device connects to that IP address

How DNS Blocking Works:

1. You type "facebook.com" in browser
2. Device asks DNS: "What's the IP address for facebook.com?"
3. DNS responds: "It's 127.0.0.1" (fake address)
4. Device tries to connect to 127.0.0.1 (which is the device itself)
5. Connection fails - website is blocked!

Limitation: DNS blocking only works for NEW requests. If a website is already cached (saved) on the device, some content might still show, but the app/website won't function properly because it can't load new content.

Slide 4: Scope Item 2 - Portal Redirection for Learning

What It Is

This slide explains how the system automatically redirects children to educational content (quizzes or videos) when their internet time expires.

Why It Matters

- Encourages learning by requiring educational activities to earn more internet time
- Prevents unlimited internet access
- Makes learning part of the internet access process

Key Concepts Explained

The Complete Flow

Step 1: Background Job Detects Time Expiration

- **Job:** `CheckTimeExpiration` runs every 2 minutes
- **What It Does:** Checks all devices to see if `remaining_time_minutes <= 0`
- **Why Every 2 Minutes:** Balances responsiveness with system performance
- **Result:** Finds devices that have run out of time

Step 2: Network-Level Blocking

- **NetworkService + iptables:** Blocks the device at the network level
- **How:** Adds DROP rules to iptables FORWARD chain
- **What This Means:** Device's MAC address is blocked - no internet traffic can pass
- **Why:** Physical blocking ensures the device truly can't access internet

Step 3: NoDogSplash Redirect

- **NoDogSplashService:** Executes redirect script
- **What It Does:**
 - Uses `ndsctl deauth <token>` command
 - Puts device in "Preauthenticated" state
 - This means NoDogSplash will intercept all HTTP requests
- **Why:** Even though device is blocked, it can still access the portal

Step 4: HTTP Request Interception

- **NoDogSplash:** Intercepts ALL HTTP requests from the device
- **What Happens:**
 - Child tries to visit any website
 - NoDogSplash intercepts the request
 - Redirects to `splash.html?tok=TOKEN`
 - Splash page then redirects to the portal

- **Why HTTP Only:** HTTPS (encrypted) traffic can't be intercepted, but most initial requests are HTTP

Step 5: Portal Interface

- **Nginx + Laravel Portal:** Serves the portal page
- **What Child Sees:**
 - Two options: "Take Quiz" or "Watch Educational Video"
 - Child selects one
 - System validates completion
 - If passed/completed: Grants more time and unblocks device
 - If failed: Child can retry or choose the other option

Why This Multi-Layer Approach?

Layer 1: iptables Blocking

- **Purpose:** Physical network-level blocking
- **Why:** Ensures device truly cannot access internet
- **Analogy:** Like a physical barrier

Layer 2: NoDogSplash Interception

- **Purpose:** Redirects HTTP requests to portal
- **Why:** Even if device tries to access internet, it gets redirected
- **Analogy:** Like a security guard redirecting you to a checkpoint

Layer 3: Portal Interface

- **Purpose:** Provides educational content
- **Why:** Children must complete learning activities to continue
- **Analogy:** Like a learning checkpoint before continuing

Key Components

iptables: The actual blocking mechanism - prevents internet access

NoDogSplash: The redirect mechanism - sends blocked devices to portal

Laravel Portal Controller: The interface - serves quiz/video pages

Nginx: The web server - hosts portal pages

Slide 5: Scope Item 3 - Schedule and Duration Management

What It Is

This slide explains how parents can set schedules (time windows) and duration limits for when children can use the internet.

Why It Matters

- Allows parents to control WHEN children can use internet (e.g., only after homework)
- Limits HOW LONG children can use internet per day
- Enforces rules automatically without constant monitoring

Key Concepts Explained

How It Works

Step 1: Parent Creates Schedule

- **Where:** Parent dashboard (web interface)
- **What They Set:**
 - **Day:** Which day of the week (Monday, Tuesday, etc.)
 - **Start Time:** When internet access begins (e.g., 3:00 PM)
 - **End Time:** When internet access ends (e.g., 9:00 PM)
 - **Duration Limit:** Maximum time allowed (e.g., 2 hours)
- **Example:** "Monday, 3:00 PM - 9:00 PM, maximum 2 hours"

Step 2: Laravel Processes Schedule

- **DeviceScheduleController:** Validates and saves schedule
- **Validation:**
 - Ensures end time is after start time
 - Ensures parent owns the device
 - Checks for time conflicts
- **Storage:** Saves to `device_schedules` table in MariaDB

Step 3: Database Storage

- **device_schedules:** Stores time windows (day, start_time, end_time, duration_limit)

- **devices:** Stores time allocation per device
- **device_time_grants:** Tracks time grants given after quiz/video completion
- **Why Multiple Tables:**
 - Schedules define rules
 - Devices store current state
 - Time grants track history

Step 4: Automatic Enforcement

- **Background Job:** `EnforceSchedules` runs every 1 minute
- **What It Does:**
 - i. Checks current time and day
 - ii. Compares with all device schedules
 - iii. If current time is OUTSIDE schedule window: Blocks device
 - iv. If current time is INSIDE schedule window: Unblocks device (if not blocked for other reasons)
 - v. Checks duration limits
- **Why Every 1 Minute:** More frequent checking for accurate schedule enforcement

Real-World Example

Scenario: Parent sets schedule "Monday, 3:00 PM - 9:00 PM, max 2 hours"

What Happens:

- **2:00 PM:** Device is blocked (before schedule starts)
- **3:00 PM:** Device is unblocked (schedule starts)
- **5:00 PM:** Device has used 2 hours, gets blocked (duration limit reached)
- **9:00 PM:** Device would be blocked anyway (schedule ends)

Key Components

MariaDB: Stores all schedule rules and time allocations

Laravel Background Jobs: `EnforceSchedules` automatically enforces rules

iptables: Physically blocks/unblocks devices based on schedule

Nginx + Laravel Dashboard: Parent interface for creating schedules

Slide 6: Scope Item 5 - Time Tracking

What It Is

This slide explains how the system tracks how much time each child device spends online and deducts it from their allocated time.

Why It Matters

- Parents need to know how much time children are actually using
- System needs to know when time expires to trigger portal redirect
- Enables fair time allocation and enforcement

Key Concepts Explained

How Time Tracking Works

Step 1: Monitor Device Connections

- **Background Job:** `MonitorDeviceConnections` runs every 2 minutes
- **What It Does:**
 - Queries ARP table (Address Resolution Protocol - shows which devices are connected)
 - Uses `get_connected_devices.sh` script
 - Identifies currently connected devices by MAC address
- **Why:** Determines which devices are actually online right now

Step 2: Track Active Sessions

- **Background Job:** `TrackActiveSessions` runs every 5 minutes
- **What It Does:**
 - i. Reads `device_sessions` table from MariaDB
 - ii. Finds active sessions (sessions that haven't ended)
 - iii. Correlates MAC addresses with database records
 - iv. Calculates how long each session has been running
 - v. Deducts time from device's `remaining_time_minutes`
 - vi. Updates `last_seen_at` timestamp
- **Example:**
 - Device had 60 minutes allocated
 - Session has been active for 15 minutes
 - System deducts 15 minutes

- Remaining time: 45 minutes

Step 3: Database Storage

- **device_sessions**: Tracks each internet session
 - `started_at` : When device started using internet
 - `ended_at` : When device stopped (NULL if still active)
 - `duration_minutes` : How long the session lasted
- **devices**: Stores `remaining_time_minutes` for each device
- **device_time_grants**: History of time grants (after quiz/video completion)

Step 4: Display to Parents

- **Nginx + Laravel Dashboard**: Shows real-time information
- **What Parents See**:
 - Current time usage for each device
 - Remaining time for each device
 - Time usage history
 - When time was granted and why

How Time Deduction Works

Example Scenario:

1. **3:00 PM**: Device connects, has 60 minutes allocated
2. **3:05 PM**: `TrackActiveSessions` runs, calculates 5 minutes used, deducts 5 minutes
 - Remaining: 55 minutes
3. **3:10 PM**: `TrackActiveSessions` runs again, calculates 5 more minutes used
 - Remaining: 50 minutes
4. **3:55 PM**: Device has used 55 minutes, remaining: 5 minutes
5. **4:00 PM**: Device has used all 60 minutes, remaining: 0 minutes
6. **4:00 PM**: `CheckTimeExpiration` detects time = 0, blocks device and redirects to portal

Key Components

iptables monitor_traffic.sh: Queries network traffic statistics

Laravel Background Jobs:

- `MonitorDeviceConnections` : Detects which devices are online
- `TrackActiveSessions` : Calculates and deducts time

MariaDB: Stores all time tracking data

Slide 7: Scope Item 7 - Parent Dashboard

What It Is

This slide explains the web-based dashboard that parents use to configure and manage the entire system.

Why It Matters

- Provides a user-friendly interface for parents
- Centralizes all system management in one place
- Makes complex network control accessible to non-technical users

Key Concepts Explained

How the Dashboard Works

Step 1: Parent Accesses Dashboard

- **URL:** `http://192.168.4.1` (the Raspberry Pi's IP address)
- **Device:** Parent's computer, tablet, or phone (on any network)
- **Authentication:** Parent must log in with username and password

Step 2: Nginx Web Server

- **What:** Web server software that handles HTTP requests
- **What It Does:**
 - Receives request from parent's browser
 - Routes request to PHP-FPM (PHP processor)
 - Serves static files (CSS, JavaScript, images)
 - Handles HTTPS/SSL encryption for security
- **Why:** Separates web server concerns from application logic

Step 3: PHP-FPM + Laravel Application

- **PHP-FPM:** FastCGI Process Manager - processes PHP code
- **Laravel:** The application framework that handles business logic

- **Controllers:** Different controllers handle different features:
 - **DeviceController:** Add, edit, block, whitelist devices
 - **DeviceScheduleController:** Create and manage schedules
 - **PortalController:** Manage captive portal (quizzes/videos)
 - **QuizController:** Create and manage quizzes
 - **VideoController:** Add and manage educational videos
 - **BlockedWebsiteController:** Block websites
 - **FlaggedWebsiteController:** Flag websites for monitoring
 - **BrowsingLogController:** View browsing history
 - **AccessAttemptController:** View security events

Step 4: MariaDB Database

- **What:** Stores all system data
- **What It Stores:**
 - Device information
 - Schedules
 - Browsing logs
 - Blocked/flagged websites
 - Quiz questions and answers
 - Video information
 - Time tracking data
- **Why:** Centralized data storage for all features

Step 5: User Interface Rendering

- **Laravel Blade Templates:** Server-side HTML rendering
- **Alpine.js:** JavaScript framework for interactive UI
- **What Parents See:**
 - Dashboard with overview of all devices
 - Forms to add/edit devices, schedules, quizzes, videos
 - Lists of browsing history, blocked websites
 - Real-time updates (via WebSockets)
- **Why:** Makes complex system management easy and intuitive

Key Features of the Dashboard

Device Management:

- View all connected devices

- Add new child devices
- Set time allocations
- Block/unblock devices
- Whitelist devices

Website Management:

- View browsing history
- Flag websites for monitoring
- Block websites (URL, domain, or app-level)
- See blocked website access attempts

Schedule Management:

- Create time-based access rules
- Set daily schedules
- Set duration limits

Content Management:

- Create quizzes with questions and answers
- Add educational videos
- Set time rewards for quiz/video completion

Monitoring:

- View real-time time usage
- See browsing history
- Monitor security events
- Track time grants

Key Components

Nginx: Web server that hosts the application

Laravel Application: Business logic and routing

MariaDB: Data persistence

Blade + Alpine.js: User interface rendering

Slide 8: Scope Item 8 - Device Management

What It Is

This slide explains how parents can manage connected devices - viewing, blocking, and whitelisting them.

Why It Matters

- Parents need to see which devices are connected
- Parents need to control which devices can access internet
- Whitelisting allows certain devices to bypass blocking rules

Key Concepts Explained

How Device Management Works

Step 1: Parent Views Connected Devices

- **Dashboard:** Parent sees list of all devices
- **Information Shown:**
 - Device name
 - MAC address (unique identifier)
 - IP address (current)
 - Connection status (online/offline)
 - Time remaining
 - Blocked/whitelisted status

Step 2: Parent Selects Action

- **Options:**
 - Block device (prevent internet access)
 - Unblock device (allow internet access)
 - Whitelist device (bypass all blocking rules)
 - Remove whitelist (subject device to blocking again)

Step 3: NetworkService Processes Request

- **What:** Laravel service that handles device operations
- **Methods:**

- `getConnectedDevices()` : Gets list of currently connected devices
- `blockDevice()` : Blocks a device
- `unblockDevice()` : Unblocks a device
- `whitelistDevice()` : Whitelists a device
- **How:** Calls shell scripts to execute network commands

Step 4: ScriptExecutor (Security Layer)

- **What:** Security wrapper that validates before executing scripts
- **What It Does:**
 - Validates script is in whitelist (only approved scripts can run)
 - Validates script path (prevents directory traversal attacks)
 - Sanitizes arguments (prevents command injection)
 - Executes script with `sudo` (elevated privileges)
 - Logs execution for audit trail
- **Why:** Prevents security vulnerabilities like command injection

Step 5: Bash Scripts Execute

- **`get_connected_devices.sh`:** Queries ARP table to find connected devices
- **`block_device.sh`:** Adds iptables DROP rules for device's MAC address
- **`unblock_device.sh`:** Removes iptables DROP rules
- **`whitelist_device.sh`:** Adds iptables ACCEPT rules at position 1 (checked before blocking rules)

Step 6: iptables Enforces Rules

- **INPUT Chain:** Blocks access to the Raspberry Pi itself
- **FORWARD Chain:** Blocks internet access (traffic between WiFi and Ethernet)
- **MAC-based Rules:** Uses device's MAC address to identify and block/unblock
- **Why MAC Address:** MAC address is unique and permanent - even if IP changes, device can be identified

Understanding MAC Addresses

What is a MAC Address?

- **MAC** = Media Access Control
- **Format:** AA:BB:CC:DD:EE:FF (6 pairs of hexadecimal characters)
- **Unique:** Every network adapter has a unique MAC address
- **Permanent:** Unlike IP addresses, MAC addresses don't change

Why Use MAC Addresses?

- **Reliable Identification:** Even if device changes IP address, MAC stays the same
- **Network-Level Control:** Can block device regardless of what IP it has
- **Persistent:** Blocking persists even if device disconnects and reconnects

Blocking vs Whitelisting

Blocking a Device:

- Adds DROP rules to iptables
- Device cannot access internet
- Device cannot access Raspberry Pi
- Can still access portal (for quiz/video)

Whitelisting a Device:

- Adds ACCEPT rules at position 1 in iptables
- Rules are checked BEFORE blocking rules
- Device bypasses ALL blocking rules
- Device always has internet access
- Useful for parent devices or trusted devices

Key Components

iptables: Physical network-level blocking via MAC address rules

ScriptExecutor: Secure execution of network control scripts

NetworkService: High-level interface for device management

Laravel Dashboard: Parent interface for device operations

Slide 9: Scope Item 9 - Security Measures

What It Is

This slide explains the security measures implemented to protect the system from unauthorized access.

Why It Matters

- Prevents unauthorized users from accessing the system
- Protects children's data and privacy

- Ensures only parents can configure the system
- Prevents malicious attacks

Key Concepts Explained

Multi-Layer Security Approach

Layer 1: Network Security (Nginx)

- **HTTPS/TLS Encryption:** All communication is encrypted
- **What:** Uses SSL/TLS certificates (RFC 8446 standard)
- **Why:** Prevents eavesdropping - even if someone intercepts traffic, they can't read it
- **Analogy:** Like sending mail in a locked, tamper-proof box

Layer 2: Application Security (Laravel)

- **Authentication:** Username and password required
- **Password Hashing:** Passwords are hashed with bcrypt (one-way encryption)
 - Even if database is compromised, passwords can't be recovered
- **CSRF Protection:** Cross-Site Request Forgery tokens on all forms
 - Prevents malicious websites from submitting forms on your behalf
- **Session Management:** Secure session storage
 - Sessions expire after inactivity
 - Sessions are tied to IP address
- **Role-Based Access Control:** Different permissions for parents vs admins
 - Parents can only manage their own devices
 - Admins have full access

Layer 3: Network-Level Security (iptables)

- **Firewall Rules:** Blocks unauthorized access attempts
- **MAC-based Filtering:** Can block devices by MAC address
- **Whitelist Rules:** Trusted devices can bypass blocking
- **Why:** Even if someone bypasses application security, network-level security provides another layer

Layer 4: Command Security (ScriptExecutor)

- **Script Whitelist Validation:** Only approved scripts can run
- **Path Validation:** Prevents directory traversal attacks
 - Example: Prevents `../../../../etc/passwd` type attacks

- **Argument Sanitization:** Prevents command injection
 - Example: Prevents `; rm -rf /` type attacks
- **Audit Logging:** All script executions are logged
 - Who ran what command, when, and why

Layer 5: Data Security (MariaDB)

- **Hashed Credentials:** Passwords are never stored in plain text
- **Session Storage:** Secure session data storage
- **Audit Logs:** Records of all important actions
- **Why:** Protects sensitive data even if database is accessed

Understanding Security Concepts

Authentication vs Authorization:

- **Authentication:** "Who are you?" (login with username/password)
- **Authorization:** "What can you do?" (permissions based on role)

Hashing vs Encryption:

- **Hashing:** One-way process (can't be reversed)
 - Used for passwords - system can verify password but can't recover it
- **Encryption:** Two-way process (can be decrypted)
 - Used for data transmission - can be decrypted with key

CSRF (Cross-Site Request Forgery):

- **What:** Attack where malicious website tricks you into submitting forms
- **Example:** Malicious site has form that submits to your dashboard
- **Protection:** CSRF tokens - unique token for each form that must match
- **Why:** Prevents unauthorized actions even if you're logged in

Key Components

Nginx: HTTPS encryption and secure transport

Laravel: Authentication, CSRF protection, session management

iptables: Network-level firewall and MAC-based filtering

ScriptExecutor: Secure command execution with validation

MariaDB: Secure credential storage and audit logging

Slide 10: Delimitation - System Constraints

What It Is

This slide explains the limitations and constraints of the system - what it CANNOT do and why.

Why It Matters

- Sets realistic expectations
- Explains why certain features aren't possible
- Shows how components work within these limitations
- Important for understanding system boundaries

Key Concepts Explained

Constraint 1: HTTPS and Encryption Limitation

What the Constraint Is:

- System cannot inspect encrypted (HTTPS) traffic
- Cannot see the actual content of encrypted websites
- Can only block at DNS level (domain names), not content level

Why This Limitation Exists:

- **HTTPS Encryption:** Modern websites use HTTPS which encrypts all content
- **Privacy by Design:** Encryption is designed to prevent inspection
- **Technical Limitation:** Without the encryption keys, content can't be decrypted

How Components Work Within This Limitation:

- **dnsmasq:** Blocks at DNS level (domain names)
 - Can block `facebook.com` but can't see what page on Facebook
 - Can't filter specific content within Facebook
- **NoDogSplash:** Only intercepts HTTP (not HTTPS) requests
 - Can redirect HTTP requests to portal
 - HTTPS requests bypass interception (by design for security)
- **Result:** Domain-level blocking works, but no deep packet inspection or content filtering

Real-World Analogy:

- Like a security guard who can see who enters a building (domain) but can't see what they do inside (encrypted content)
- Can block someone from entering, but can't monitor their activities inside

Constraint 2: Limited Processing Capacity

What the Constraint Is:

- Raspberry Pi has limited CPU and RAM compared to servers
- Cannot handle computationally intensive tasks
- Must optimize for efficiency

Why This Limitation Exists:

- **Hardware Limits:** Raspberry Pi 4B has 4GB RAM and quad-core CPU
- **Cost Constraint:** More powerful hardware would increase cost
- **Power Consumption:** Must run 24/7, so efficiency matters

How Components Work Within This Limitation:

- **Laravel Background Jobs:** Run at intervals (1-10 minutes), not continuously
 - Reduces CPU usage
 - Balances responsiveness with performance
- **iptables:** Lightweight firewall rules
 - No deep packet inspection (which requires more processing)
 - Simple rule matching is fast
- **dnsmasq:** Efficient DNS blocking
 - Simple domain matching, no content analysis
 - Very fast even with many blocked domains
- **Result:** System works well for its purpose, but can't do heavy analytics

Real-World Analogy:

- Like a small car - efficient and does the job, but can't tow a trailer
- Optimized for the specific task (parental control), not general-purpose computing

Constraint 3: Network Infrastructure Dependencies

What the Constraint Is:

- System depends on ISP router compatibility
- Requires router that supports certain configurations

- May need router configuration assistance

Why This Limitation Exists:

- **Router Variations:** Different routers have different capabilities
- **Firmware Differences:** Some routers don't support required features
- **Network Setup:** Depends on how home network is configured

How Components Work Within This Limitation:

- **hostapd:** WiFi AP functionality requires compatible router
 - Some routers interfere with access point mode
 - May need router configuration changes
- **dnsmasq:** DHCP service requires proper network configuration
 - Router must allow Pi to act as DHCP server
 - May need to disable router's DHCP or configure properly
- **Result:** System works best with compatible routers, may need setup assistance

Real-World Analogy:

- Like a car that works best with certain types of fuel
- Works with most setups, but some configurations need adjustment

Constraint 4: Data Privacy Constraints

What the Constraint Is:

- System collects only necessary data
- No packet payload inspection
- No content logging

Why This Limitation Exists:

- **Privacy by Design:** Respects user privacy
- **Legal Compliance:** Avoids collecting unnecessary personal data
- **Ethical Considerations:** Balances monitoring with privacy

How Components Work Within This Limitation:

- **MariaDB:** Stores only:
 - Domain names (not full URLs with parameters)
 - Timestamps
 - MAC addresses

- No personal information from websites
- **Laravel ParseNetworkLogs:** Logs domain-level access only
 - Records "[facebook.com](https://www.facebook.com) was visited"
 - Does NOT record "what page on Facebook" or "what was posted"
- **Result:** System monitors activity without invading privacy

Real-World Analogy:

- Like a security guard who notes who enters a building but doesn't read their private messages
- Monitors for safety without being invasive

Constraint 5: Free and Open-Source Tools Only

What the Constraint Is:

- System uses only free and open-source software
- No commercial licensing
- No enterprise-grade features from commercial solutions

Why This Limitation Exists:

- **Cost Constraint:** Project must be affordable
- **Open-Source Philosophy:** Promotes transparency and community
- **Educational Purpose:** Students can learn from and modify code

How Components Work Within This Limitation:

- **All Components Are Open-Source:**
 - Laravel (MIT License)
 - MariaDB (GPL License)
 - Nginx (BSD License)
 - NoDogSplash (GPL License)
 - iptables (GPL License)
 - dnsmasq (GPL License)
- **Result:** System is free to use and modify, but may lack some enterprise features

Real-World Analogy:

- Like using free, community-built tools instead of expensive professional tools
- Does the job well, but may not have all the bells and whistles of commercial solutions

Slide 11: Component Roles Summary

What It Is

This slide provides a comprehensive summary of all system components, their roles, and which scope items they support.

Why It Matters

- Gives complete overview of system architecture
- Shows how each component contributes to achieving scope items
- Helps understand the "big picture" of how everything works together

Key Concepts Explained

Component Categories

Application Layer (Business Logic):

- **Laravel**: The "brain" - coordinates everything
- **MariaDB**: The "memory" - stores all data
- **PHP-FPM**: The "processor" - executes PHP code

Web Server Layer:

- **Nginx**: The "receptionist" - handles web requests

Network Control Layer:

- **iptables**: The "security guard" - blocks/unblocks at network level
- **NAT (MASQUERADE)**: The "translator" - enables network communication
- **hostapd**: The "WiFi creator" - creates the WiFi network
- **dnsmasq**: The "address book and blocker" - DNS and DHCP services

Portal Layer:

- **NoDogSplash**: The "redirector" - sends devices to portal

Service Layer:

- **DomainBlockingService**: The "app blocker" - handles app-level blocking
- **ScriptExecutor**: The "security wrapper" - safely executes scripts

Automation Layer:

- **Cron/Laravel Scheduler:** The "timer" - schedules background jobs
- **Background Jobs:** The "workers" - automated monitoring tasks
- **Systemd:** The "service manager" - keeps services running
- **Queue Worker:** The "job processor" - executes background jobs

How Components Work Together

Example: Blocking a Website

1. **Parent** uses **Nginx + Laravel Dashboard** to block facebook.com
2. **Laravel** processes request via **BlockedWebsiteController**
3. **DomainBlockingService** detects related domains (30+ for Facebook app)
4. **ScriptExecutor** safely executes `block_domain.sh` script
5. **dnsmasq** updates DNS blocklist configuration
6. **dnsmasq** restarts to apply changes
7. **MariaDB** stores blocking information
8. **Background Job (ParseNetworkLogs)** monitors and logs access attempts
9. **iptables** may also block at network level if needed

Example: Time Expiration and Portal Redirect

1. **Background Job (CheckTimeExpiration)** detects time = 0
2. **Laravel** calls **NetworkService** to block device
3. **ScriptExecutor** safely executes `block_device.sh`
4. **iptables** adds DROP rules for device's MAC address
5. **NoDogSplashService** redirects device to portal
6. **NoDogSplash** intercepts HTTP requests
7. **Nginx + Laravel Portal** serves quiz/video interface
8. **MariaDB** stores quiz attempts/video completions
9. **TimeGrantingService** grants time after completion
10. **NetworkService** unblocks device
11. **iptables** removes DROP rules

Understanding Component Relationships

Laravel is the Central Orchestrator:

- All other components are tools that Laravel uses
- Laravel doesn't directly control hardware - it uses scripts and services

- Think of Laravel as the "conductor" of an orchestra

Services Provide Abstractions:

- **NetworkService**: Hides complexity of iptables commands
- **DomainBlockingService**: Hides complexity of dnsmasq configuration
- **NoDogSplashService**: Hides complexity of NoDogSplash commands
- **ScriptExecutor**: Provides security layer for all script execution

Background Jobs Provide Automation:

- Run continuously without human intervention
- Handle time-sensitive tasks (time tracking, schedule enforcement)
- Process data in background (log parsing)

Systemd Provides Reliability:

- Keeps services running 24/7
- Auto-restarts if service crashes
- Manages service dependencies

Slide 12: Questions & Discussion

What It Is

The final slide that opens the floor for questions and provides references to detailed documentation.

Why It Matters

- Allows audience to ask clarifying questions
- Provides resources for deeper learning
- Shows that comprehensive documentation exists

Key Documentation References

Network Control System Architecture:

- Detailed explanation of how network control works
- Shell scripts, PHP services, and their interactions

- Reference: NETWORK_CONTROL_SYSTEM_ARCHITECTURE.md

NoDogSplash Setup and Integration:

- How captive portal is configured
- Integration with Laravel
- Reference: NODOGSPLASH_SETUP.md , NODOGSPLASH_INTEGRATION.md

Background Jobs Overview:

- Explanation of all 5 background jobs
- How scheduling works
- Reference: BACKGROUND_JOBS_OVERVIEW.md

Device Management Implementation:

- How device management works
- Testing procedures
- Reference: DEVICE_MANAGEMENT_IMPLEMENTATION.md

General Concepts Explained

What is a Raspberry Pi?

- **What:** A small, affordable computer (about the size of a credit card)
- **Why Used:**
 - Low cost
 - Low power consumption (can run 24/7)
 - Can run Linux operating system
 - Has WiFi and Ethernet capabilities
- **In This Project:** Acts as the central control point for all child devices

What is Laravel?

- **What:** A PHP web application framework
- **Why Used:**
 - Makes web development easier and faster
 - Provides built-in security features

- Has excellent documentation
- Large community support
- **In This Project:** The "brain" that coordinates all system components

What is a Captive Portal?

- **What:** A web page that users are redirected to before accessing the internet
- **Common Example:** Hotel WiFi - when you connect, you see a login page
- **In This Project:** When time expires, children are redirected to quiz/video portal

What is a Background Job?

- **What:** Code that runs automatically in the background
- **Why Used:**
 - Automates repetitive tasks
 - Runs without user interaction
 - Handles time-sensitive operations
- **In This Project:**
 - Checks if time expired (every 2 minutes)
 - Tracks time usage (every 5 minutes)
 - Enforces schedules (every 1 minute)
 - Monitors devices (every 2 minutes)
 - Parses logs (every 10 minutes)

What is DNS?

- **What:** Domain Name System - translates website names to IP addresses
- **Example:** "google.com" → "142.250.191.14"
- **Why Important:**
 - Humans remember names better than numbers
 - DNS blocking can prevent access to websites
- **In This Project:** dnsmasq provides DNS service and blocks domains

What is DHCP?

- **What:** Dynamic Host Configuration Protocol - automatically assigns IP addresses
- **Why Important:**
 - Without it, you'd have to manually set IP on every device
 - Makes connecting devices easy

- **In This Project:** dnsmasq assigns IP addresses (192.168.4.x) to child devices

What is a MAC Address?

- **What:** Media Access Control Address - unique identifier for network adapters
- **Format:** AA:BB:CC:DD:EE:FF (6 pairs of hexadecimal characters)
- **Why Important:**
 - Permanent identifier (doesn't change like IP addresses)
 - Used to identify and block specific devices
- **In This Project:** Used to block/unblock devices regardless of IP address

Summary

This presentation explains how a complex parental control system works by breaking it down into understandable components. Each component has a specific role, and together they achieve all 9 scope items:

1. **Website Monitoring and Blocking** - dnsmasq, DomainBlockingService, ParseNetworkLogs
2. **Portal Redirection** - NoDogSplash, iptables, CheckTimeExpiration
3. **Schedule Management** - EnforceSchedules, DeviceScheduleController
4. **Time Tracking** - TrackActiveSessions, MonitorDeviceConnections
5. **Parent Dashboard** - Nginx, Laravel, MariaDB
6. **Device Management** - NetworkService, iptables, ScriptExecutor
7. **Security** - Multiple layers (Nginx, Laravel, iptables, ScriptExecutor)

The system uses open-source tools running on a single Raspberry Pi 4B, providing comprehensive parental control within the project's constraints and limitations.